

Fundsroom ERP & CRM

System Architecture & Deployment Documentation

Prepared for: Technical Evaluation Board
Author: Lead Developer Agent
Date: August 2026
Version: 1.0.0 (Production-Ready)

1. Architecture & System Overview

The Fundsroom Operations Portal is built using a decoupled Monolith Split architecture, separating a JSON REST API and a Single Page Application frontend. This layout maximizes response speeds and guarantees clear division of concerns.

Backend API Component

The backend layer is engineered in Node.js and Express using TypeScript. Incoming requests are managed through controllers, which perform schema-level validation using Zod before engaging the database client. JWT hashing guards endpoints, enforcing role-based permissions.

Frontend Web Client

The frontend is built in React and Vite using TypeScript. Styling is constructed in modular Vanilla CSS using custom design tokens to present a high-contrast corporate light theme. Session context tracks logged-in users, dynamically displaying permitted modules.

Database & ORM Model

Data is housed in a relational MySQL/MariaDB database. The schema is integrated using Prisma ORM. A MariaDB driver adapter handles connection pooling, and transactional statements guarantee atomicity during sales challan fulfillment.

2. Database Schema & Setup

The SQL schema represents relational records for internal operations. Database interactions utilize connection adapters for maximum performance.

Database Entities (Models)

- **User:** Internal employee accounts containing email, role classification, and cryptographically hashed passwords.
- **Customer:** CRM directory tracking wholesale distributors, contact information, status, and scheduled follow-ups.
- **FollowUp:** CRM notes timeline referencing logs created during customer outreach and sales cycles.
- **Product:** Inventory catalog containing SKU codes, current stock counters, alert markers, and warehouse coordinates.
- **StockMovement:** Detailed stock ledgers logging manual increments, decrements, and order releases.
- **Challan:** Sales records tracking client relations, status state transitions, and total calculations.
- **ChallanProduct:** Price-fixed transaction snapshots preserving details of sold items.

Seeding Initial Data

A seeding script is provided to populate initial database records. This loads four default employees (representing each user role), ten initial catalog products, and five wholesale customer profiles. To execute seeding in any environment, run the database seed utility.

3. Environment Variables

Environment variables govern server ports, session security, database connection details, and runtime configurations. Both local dotenv files and cloud hosting providers use these variables.

Variable Specifications

- **PORT:** The network port the Express API listens on (default: 5000).
- **DATABASE_URL:** The MySQL database connection string. Auto-parsed by the Prisma engine.
- **DB_HOST / DB_USER / DB_PASSWORD:** Individual DB connection variables used as fallback values by the MariaDB driver.
- **DB_PORT / DB_NAME / DB_SSL:** SSL activation config parameters (enable DB_SSL=true for secure cloud databases).
- **JWT_SECRET:** Cryptographic secret key used to sign and verify JWT authentication tokens.
- **NODE_ENV:** Runtime environment classifier (development or production).

Auto-Parsing Credentials

To simplify cloud deployments on serverless environments like Vercel, the application includes an auto-parsing connection utility. If DATABASE_URL is defined, the system automatically extracts connection details and enables secure SSL tunnels, eliminating the need to set individual variables.

4. Local Setup Guide

1. Database Preparation

Ensure a MySQL or MariaDB instance is running locally on port 3306. Connect and execute:
`CREATE DATABASE fundsroom_erp;`

2. Backend Setup

Navigate to `/backend`, install dependencies, configure your `.env` file, apply migrations, seed the database, and start the development server:

```
cd backend
npm install
npx prisma migrate dev --name init
npm run seed
npm run dev
```

3. Frontend Setup

Navigate to `/frontend`, install dependencies, and launch the Vite development server:

```
cd frontend
npm install
npm run dev
```

5. Features, Status & Assumptions

Feature Progress Checklist

- **Role-Based Auth (100% Complete):** Allows secure token authentication and restricts page navigation according to user roles.
- **CRM Customer Directory (100% Complete):** Supports text searches, status categorization filters, detail logs, and follow-up updates.
- **Inventory Catalog (100% Complete):** Supports item CRUD, manual stock adjustments, reorder alerts, and global ledgers.
- **Sales Challan Billing (100% Complete):** Features draft saving, transactional stock allocation, and printable invoices.
- **Docker Deployment (100% Complete):** Includes Dockerfile configurations for backend and frontend.
- **API Collection (100% Complete):** Includes a pre-configured Postman JSON collection for route testing.

Key Design Assumptions

- **Pricing Integrity:** Product names, SKUs, and pricing details are copied to snapshot records when confirmed. This ensures that future catalog changes do not alter historical invoices.
- **Stock Atomicity:** Stock decrements are validated and run within single SQL transactions to prevent negative stock quantities.
- **Single-Page Routing:** Uses modular states in the React shell to coordinate pages, optimizing load speeds.

6. Testing Credentials & Endpoints

Employee Login Profiles

Use these accounts to test the system's role-based permissions and interface modules:

- **System Admin:** Email: admin@fundsroom.com | Password: admin123 (Full Access)
- **Sales Executive:** Email: sales@fundsroom.com | Password: sales123 (CRM, Challan Drafts)
- **Warehouse Keeper:** Email: warehouse@fundsroom.com | Password: warehouse123 (Inventory CRUD, Logs)
- **Accounts Manager:** Email: accounts@fundsroom.com | Password: accounts123 (Invoices, Read-only CRM/Inventory)

API Core Endpoints (Postman)

- **POST /api/auth/login:** Authenticates user and returns JWT token.
- **GET /api/customers:** Lists clients with search and filter parameters.
- **POST /api/customers/:id/followups:** Adds a timeline note to a customer's profile.
- **POST /api/products/:id/adjust-stock:** Logs a manual stock adjustment (IN/OUT).
- **POST /api/challans:** Saves a challan (decrements stock if status is CONFIRMED).
- **PUT /api/challans/:id/status:** Updates status (transfers stock on CONFIRM/CANCEL transitions).

